ChatGPT said:

Virtual Threads in Java: DSA, LLD, and HLD Perspectives

Java Virtual Threads, introduced as a preview feature in Java 19 and finalized in **Java 21**, fundamentally change how Java applications handle concurrency. They make it practical to create very large numbers of lightweight threads without requiring one operating-system thread for every concurrent task.

For a senior software engineer or architect, the important question is not just how to create virtual threads. It is how they affect scheduling, resource management, synchronization, API design, throughput, and distributed-system architecture.

We'll approach the topic from three perspectives:

1. DSA — Data Structures and Algorithms

Thread scheduling, queues, complexity, contention, throughput, and the relationship between concurrency and parallelism.

2. LLD — Low-Level Design

ExecutorService, synchronization, thread safety, resource limits, structured concurrency, and implementation patterns.

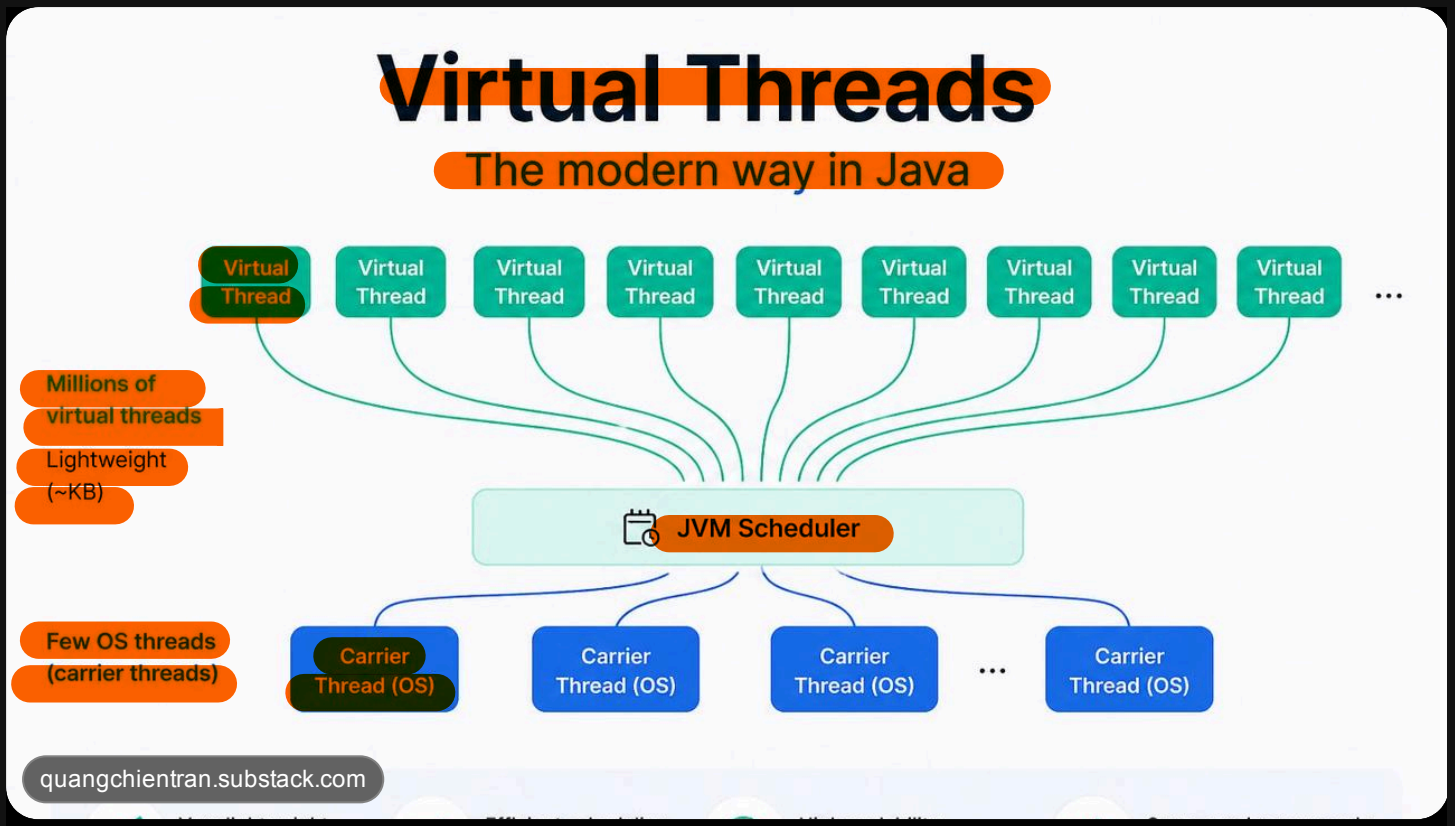
3. HLD — High-Level Design

Web services, database connection pools, downstream APIs, backpressure, capacity planning, and production architecture.

1. Fundamentals: What is a Virtual Thread?

A traditional Java platform thread is backed by an operating-system thread. Virtual threads are lightweight Java-managed threads that the JVM schedules onto a smaller set of platform threads.

The platform threads used to execute virtual threads are commonly called **carrier threads**.



Platform threads versus virtual threads

Property	Platform thread	Virtual thread
Execution	Backed by an OS thread	Scheduled onto a carrier OS thread
Memory cost	Generally higher per thread	Generally much lower per thread
Creation	Relatively expensive at large scale	Relatively inexpensive
Blocking I/O	Can occupy an OS thread while blocked	Often releases the carrier thread
Best suited for	CPU-intensive tasks and conventional concurrency	Large numbers of I/O-bound tasks
Parallel execution	Yes, across CPU cores	Yes, through carrier threads across CPU cores
Thread pooling	Often useful	Usually unnecessary for virtual threads themselves

The key distinction is that **virtual threads reduce the cost of waiting; they do not make CPU execution free or increase the number of CPU cores.**

A simple example

```
Java
```

```
public class VirtualThreadDemo {
    public static void main(String[] args)
            throws InterruptedException {

        Thread thread = Thread.ofVirtual().start(() -> {
            System.out.println(
                "Running on: " + Thread.currentThread()
            );
        });

        thread.join();
    }
}
```

You can also use a virtual-thread-per-task executor:

Java

```
import java.util.concurrent.*;

public class Main {
    public static void main(String[] args) throws Exception {
        try (var executor =
                 Executors.newVirtualThreadPerTaskExecutor()) {

            Future<String> result = executor.submit(() -> {
                Thread.sleep(1000);
                return "Request completed";
            });

            System.out.println(result.get());
        }
    }
}
```

Each submitted task gets its own virtual thread. The executor does not behave like a fixed-size pool that reuses a limited number of virtual threads.

Java 21 or later is required for these finalized APIs.

---

2. DSA perspective: Scheduling, complexity, and contention

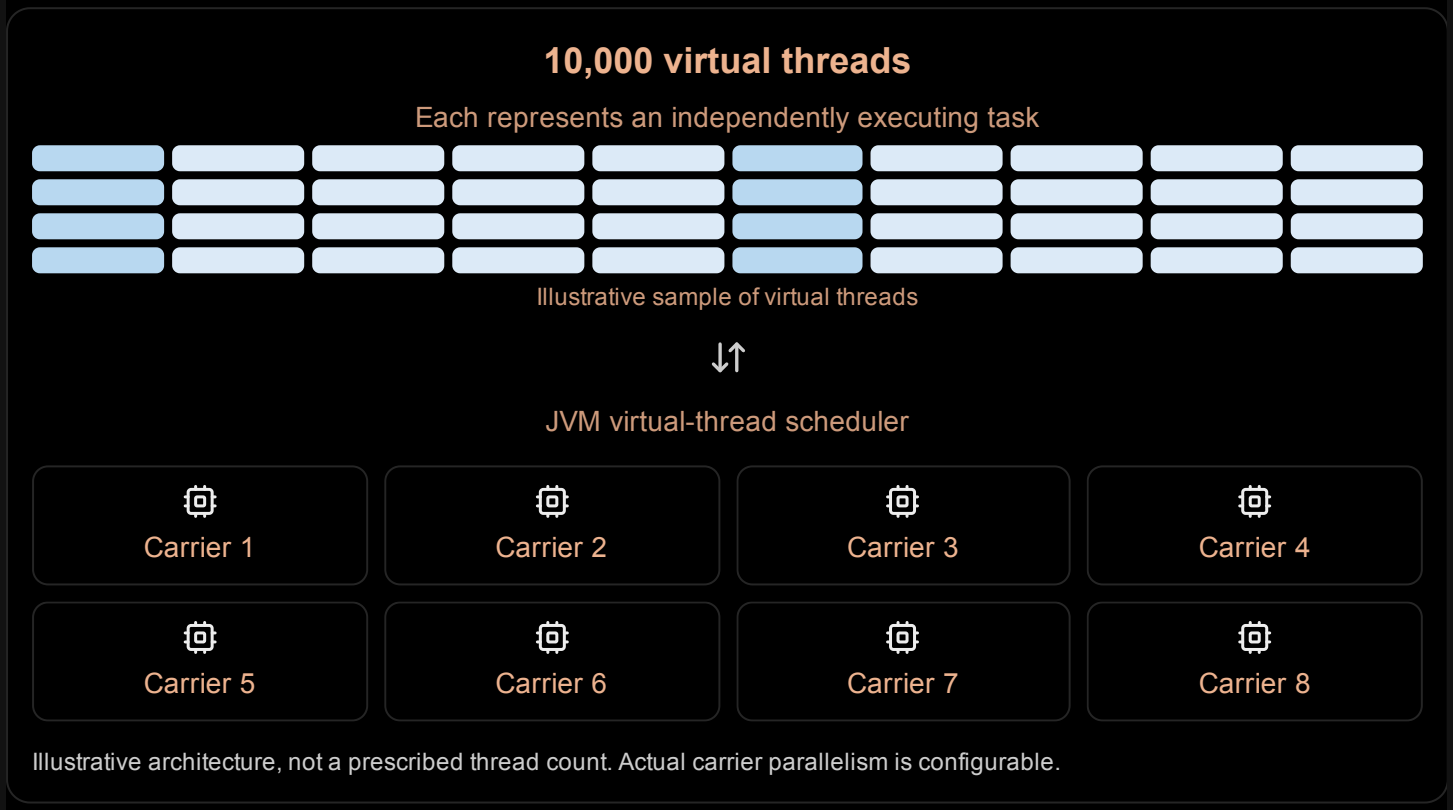
Virtual threads are not a new data structure or algorithm. Their significance in DSA comes from how they change the cost of concurrent execution and the bottlenecks that determine throughput.

2.1 Understand M:N scheduling

The JVM uses an M:N scheduling model:

- **M:** a large number of virtual threads.
- **N:** a smaller number of carrier platform threads.
- The JVM scheduler assigns virtual threads to carriers; the operating system schedules carriers onto CPU cores.

Consider a machine with 8 available processors.



The important optimization happens when a virtual thread blocks on a supported operation, such as a socket read. The JVM can suspend that virtual thread and let its carrier execute another task.

When the I/O operation completes, the virtual thread becomes eligible to resume.

Important: a virtual thread can execute on different carriers during its lifetime. It is not permanently associated with one carrier.

2.2 Concurrency versus parallelism

These concepts are frequently confused in interviews.

- **Concurrency:** multiple tasks make progress during overlapping periods.
- **Parallelism:** multiple tasks execute simultaneously on different CPU cores.

Virtual threads make high concurrency inexpensive. They do not guarantee more parallel execution.

Suppose a service receives 1,000 requests, and each request performs 10 ms of CPU work and then waits 490 ms for an external service.

Assuming the CPU and external service are not bottlenecks:

- Each request takes approximately 500 ms.
- Each request consumes only about 10 ms of CPU.
- Most of its lifetime is spent waiting.

Virtual threads allow many requests to remain in progress without tying up an operating-system thread for the entire 500 ms.

2.3 Little's Law: A key HLD/DSA connection

Little's Law relates throughput, concurrency, and latency:

$$L = \lambda W$$

Where:

- L = average number of requests in the system.
- λ = average throughput in requests per second.
- W = average time in the system in seconds.

For example, if a service processes 2,000 requests per second with an average latency of 0.5 seconds:

$$L = 2000 \times 0.5 = 1000$$

The service needs to support approximately 1,000 requests in flight on average.

A conventional thread-per-request model with platform threads can become expensive at this concurrency level. Virtual threads make it more practical to retain the simple thread-per-request programming model.

However, Little's Law does not mean that increasing concurrency automatically increases throughput. Throughput may remain capped by a database, CPU, or downstream service.

2.4 CPU-bound versus I/O-bound complexity

Consider two workloads:

Workload	Dominant cost	Expected benefit from virtual threads

Waiting for HTTP responses	Network latency	High potential
Waiting for database queries	Database latency	High potential, if the connection pool can support the workload
Reading sockets	I/O latency	High potential
JSON parsing	CPU instructions	Usually limited
Cryptographic hashing	CPU instructions	Usually limited
Matrix multiplication	CPU instructions	Usually limited

The time complexity of an algorithm does not change just because it runs on a virtual thread.

For example, sorting n elements remains $O(n \log n)$. Virtual threads do not improve that algorithm's asymptotic complexity.

They improve how efficiently many concurrent tasks share execution resources, particularly while waiting.

2.5 Queueing and contention

Virtual threads can make it easy to create thousands of tasks, but those tasks may still contend for shared resources.

Consider:

Java

```
private static final Object LOCK = new Object();

static void processRequest() {
    synchronized (LOCK) {
        callRemoteService();
    }
}
```

All requests must acquire the same lock before calling the remote service. If the remote call takes 200 ms, the critical section serializes access and limits throughput, regardless of how many virtual threads exist.

If one shared lock protects a 200 ms operation, its idealized maximum service rate is approximately:

$$\frac{1}{0.2} = 5 \text{ operations/second}$$

Virtual threads do not remove this bottleneck.

The same reasoning applies to bounded queues, semaphores, database connections, and rate limits.

DSA takeaway: learn to identify the true limiting resource. Virtual threads reduce thread scarcity; they do not eliminate algorithmic complexity, lock contention, or queueing bottlenecks.

3. LLD perspective: Implementing virtual-thread-based services

At the low-level design layer, virtual threads affect how you structure task execution, synchronization, cancellation, and resource protection.

3.1 Choosing the right execution model

Java provides several execution models.

API	Intended use
`Thread.ofVirtual()`	Create an individual virtual thread
`Executors.newVirtualThreadPerTaskExecutor()`	Run each task on a new virtual thread
`Executors.newFixedThreadPool(n)`	Reuse a bounded number of platform threads by default
`Executors.newWorkStealingPool()`	Parallel task processing, typically for CPU-oriented decomposition
`CompletableFuture`	Compose asynchronous stages and results

For a conventional I/O-heavy service, virtual-thread-per-task execution is a natural starting point.

Java

```
try (var executor =
        Executors.newVirtualThreadPerTaskExecutor()) {

    Future<String> user = executor.submit(
        () -> userService.fetchUser("123")
    );

    Future<String> orders = executor.submit(
        () -> orderService.fetchOrders("123")
    );

    System.out.println(user.get());
```

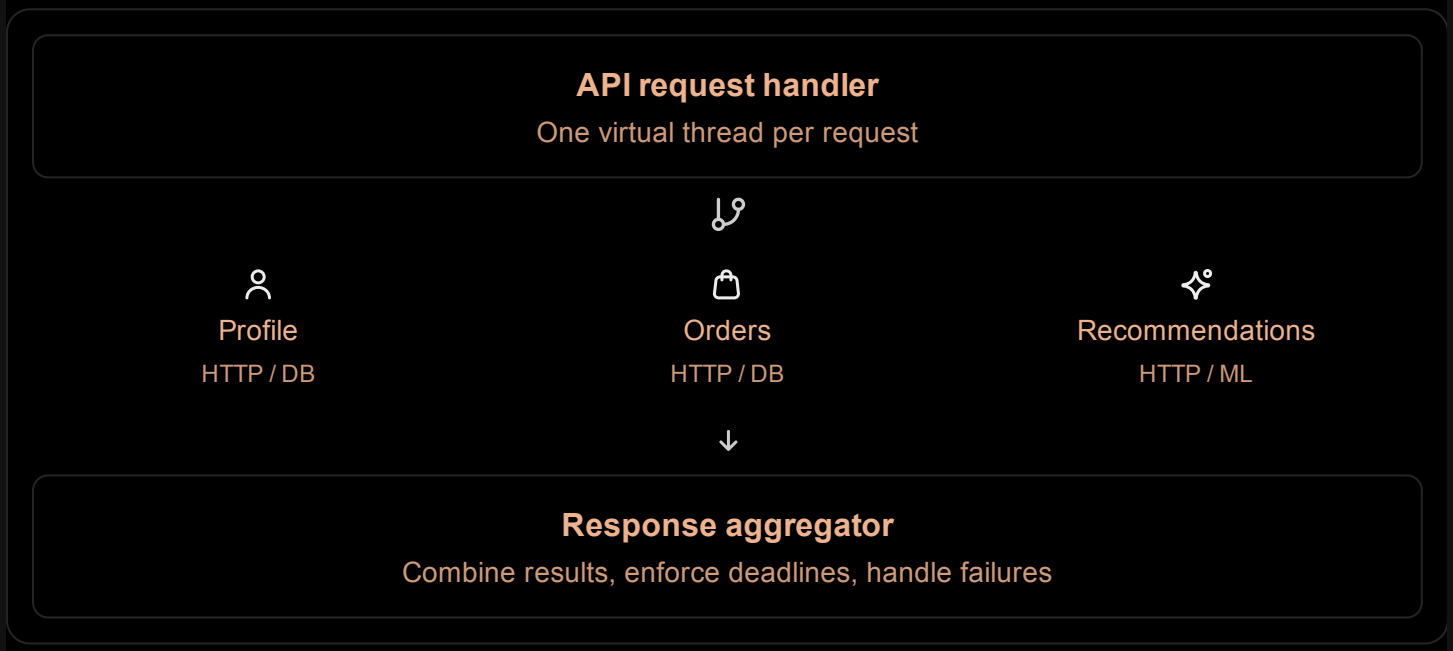
```
    System.out.println(orders.get());
}
```

The two operations can proceed concurrently. Calling `get()` blocks the current thread until the result is available; if that thread is virtual, a supported blocking wait can release its carrier.

The executor's `close()` waits for submitted tasks to finish. It does not automatically cancel every task just because the calling thread no longer needs the result.

3.2 Design a concurrent aggregator

Imagine an API endpoint that returns a customer's profile, orders, and recommendations.



An implementation can use an executor and futures:

Java

```
import java.util.concurrent.*;

record CustomerResponse(
    String profile,
    String orders,
    String recommendations
) {}

class CustomerFacade {
    private final ProfileService profiles;
    private final OrderService orders;
    private final RecommendationService recommendations;

    CustomerFacade(
```

```
            ProfileService profiles,
            OrderService orders,
            RecommendationService recommendations) {
        this.profiles = profiles;
        this.orders = orders;
        this.recommendations = recommendations;
```

This example illustrates concurrent fan-out. A production implementation should also define:

- A request deadline and downstream timeouts.
- What happens when one dependency fails.
- Whether some fields are optional.
- How sibling tasks are cancelled when their results are no longer needed.
- How downstream concurrency is limited.

The try-with-resources block waits for all submitted tasks to finish before returning. If one future fails, other tasks may still be running while the executor is closed. A real implementation needs deliberate failure and cancellation semantics.

3.3 Synchronization and thread safety

Virtual threads are still Java threads. The usual Java Memory Model rules continue to apply.

This code has a race condition:

Java

```
class Counter {
    private int count;

    void increment() {
        count++;
    }

    int getCount() {
        return count;
    }
}
```

If many virtual threads invoke `increment()` concurrently, increments can be lost.

One correction is:

Java

```
class Counter {
    private final java.util.concurrent.atomic.AtomicInteger count =
        new java.util.concurrent.atomic.AtomicInteger();

    void increment() {
        count.incrementAndGet();
    }

    int getCount() {
        return count.get();
    }
}
```

For compound state transitions, a lock or other appropriate synchronization mechanism may be necessary. Virtual threads do not make shared mutable state safe.

3.4 The `synchronized` and pinning version trap

Pinning occurs when a virtual thread cannot unmount from its carrier during a blocking operation. The exact implications depend on the JDK version.

- **Java 21–23:** blocking inside `synchronized` code could pin a virtual thread to its carrier.
- **Java 24 and later:** JEP 491 removes the monitor-related pinning limitation for `synchronized` methods and blocks.
- **Native methods and foreign-function calls:** can still pin virtual threads in relevant circumstances.

Therefore, do not blindly replace every `synchronized` block with `ReentrantLock`. Keep critical sections short, avoid holding locks across remote calls, and investigate actual pinning with profiling and JFR.

openjdk.org +2

3.5 Backpressure and resource limits

This is one of the most important LLD lessons.

Java

```
try (var executor =
         Executors.newVirtualThreadPerTaskExecutor()) {

    for (int i = 0; i < 100_000; i++) {
        executor.submit(() -> callPartnerApi());
```

```
    }
}
```

This code can create a very large number of virtual threads. It does not, however, enforce a safe limit on concurrent partner API calls.

Use a semaphore or another explicit admission-control mechanism when you need to limit concurrency:

Java

```java
import java.util.concurrent.Semaphore;

class PartnerClient {
    private final Semaphore permits = new Semaphore(100);

    String call() throws Exception {
        permits.acquire();

        try {
            return performRemoteCall();
        } finally {
            permits.release();
        }
    }

    private String performRemoteCall() {
        // Invoke the actual HTTP client here.
        return "OK";
    }
}
```

Here, at most 100 calls can hold permits at once. Other virtual threads wait for a permit, usually without occupying a carrier while parked.

In a real service, acquire permits with an appropriate timeout if indefinite waiting is unacceptable. Also account for every application instance when sizing the downstream concurrency limit.

3.6 ThreadLocal considerations

Virtual threads support `ThreadLocal`, which helps existing code work with less modification. However, per-thread state that was reasonable for a small platform-thread pool can become expensive when millions of virtual threads are created.

Be particularly careful with:

- Large objects stored in thread locals.
- Thread-local database or HTTP clients.
- Per-thread caches.
- Request context and security context propagation.
- Cleanup when tasks finish.

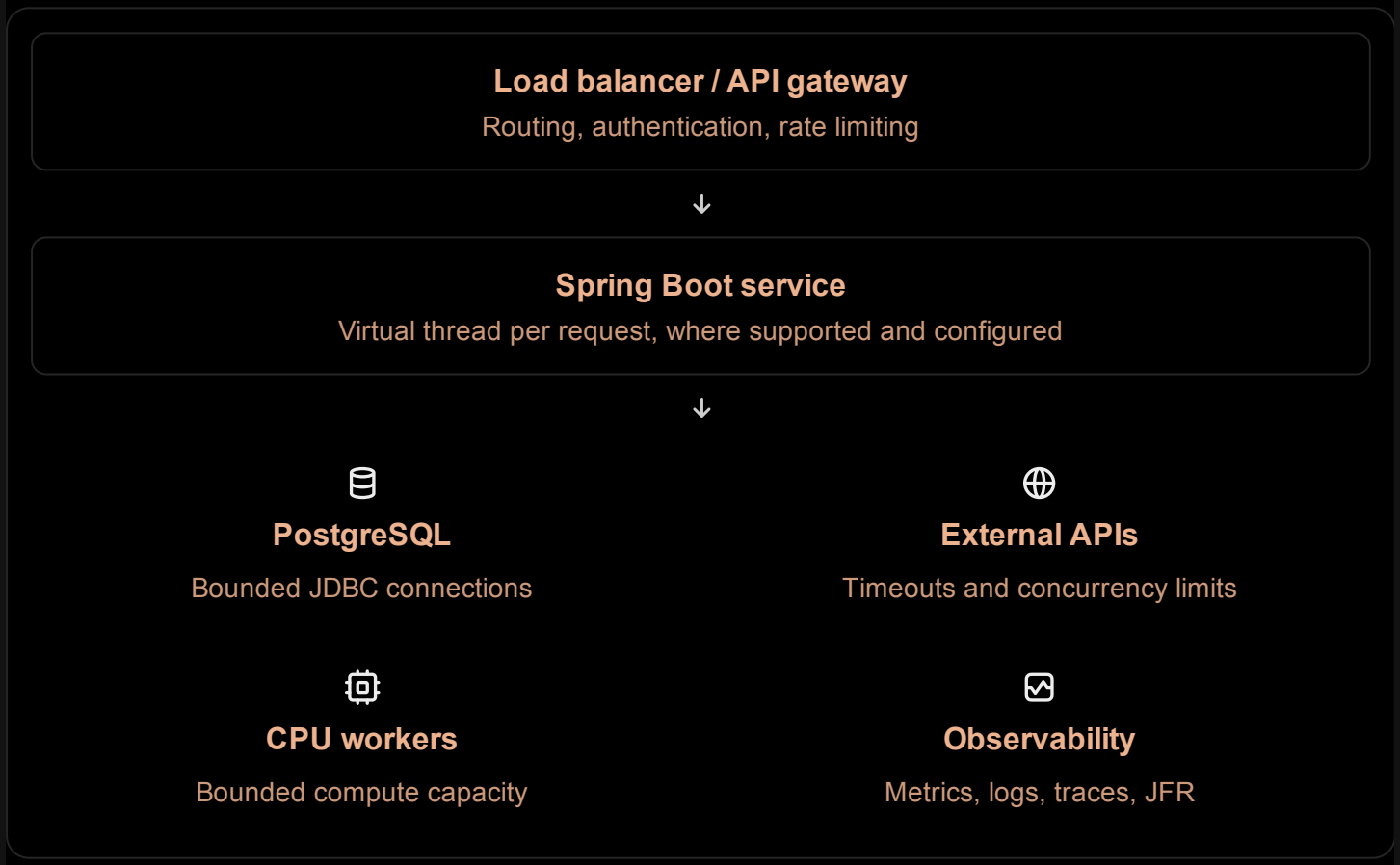
Prefer shared, thread-safe clients and explicit resource management when possible. Scoped values are also worth studying for suitable immutable contextual data.

---

4. HLD perspective: Designing scalable backend systems

At the high-level design layer, virtual threads change the execution model of a service, but they do not eliminate the need for capacity planning.

Consider a typical Spring Boot microservice:



4.1 The database connection pool problem

Suppose your service handles 5,000 concurrent requests, and every request needs a PostgreSQL query.

Your HikariCP pool is configured with 50 connections.

The system cannot execute 5,000 database queries simultaneously through that pool. At most 50 connections can be checked out at once, assuming each connection is dedicated to one active operation.

The remaining requests must wait for a connection or be rejected or timed out according to your configuration.

Virtual threads make waiting for connections less expensive, but they do not increase database capacity.

A rough theoretical upper bound, assuming all connections are continuously busy and each query takes 20 ms, is:

$$\text{Throughput} \approx \frac{50}{0.020} = 2500 \text{ queries/second}$$

This is an idealized bound, not a performance prediction. Real throughput depends on database execution time, transaction duration, contention, connection overhead, and other workload characteristics.

Architectural lesson: size the database pool according to the database's sustainable capacity, not according to the number of virtual threads.

4.2 Avoid thread pools as the default solution to virtual-thread overload

With traditional platform threads, a fixed thread pool commonly serves two purposes:

1. Reusing expensive threads.
2. Limiting the number of concurrently executing tasks.

With virtual threads, thread reuse is generally unnecessary. But resource protection is still essential.

Separate these concerns:

Concern	Recommended mechanism
Execute blocking request tasks	Virtual-thread-per-task executor
Limit concurrent database operations	Connection pool and, where needed, admission control
Limit external API calls	Semaphore, rate limiter, or bounded work queue
Limit CPU-heavy work	Bounded platform-thread executor
Limit inbound requests	Gateway or server admission control
Limit total memory pressure	Request-size limits, concurrency budgets, and load shedding

A semaphore limits concurrency, whereas a rate limiter controls the number of operations over time. They solve related but different problems.

4.3 CPU-heavy workloads need a different design

Suppose an endpoint performs substantial image processing, cryptographic calculations, or complex data transformations.

Creating 100,000 virtual threads does not create 100,000 CPU cores. If those tasks all need CPU time, they compete for the same processors.

A better design separates I/O orchestration from compute execution:

Java

```
private final ExecutorService ioExecutor =
    Executors.newVirtualThreadPerTaskExecutor();

private final ExecutorService cpuExecutor =
    Executors.newFixedThreadPool(
        Runtime.getRuntime().availableProcessors()
    );
```

Use the virtual-thread executor for blocking I/O and the bounded compute executor for CPU-intensive tasks.

The exact CPU pool size should be established through benchmarking. In some workloads, a pool slightly larger than the processor count may be appropriate; in others, fewer threads may perform better.

Also note that an executor does not automatically route nested work to the other executor. Your code must explicitly submit compute tasks to `cpuExecutor`, and you must manage the lifecycle of both executors.

4.4 Deadlines, timeouts, and cancellation

In a distributed system, a request may traverse several services:

Plain text

```
Client
  -> API Gateway
      -> Order Service
          -> Inventory Service
```

```
            -> Payment Service
            -> Shipping Service
```

Virtual threads make it easier to write sequential-looking code that waits for each dependency. They do not make an unbounded wait safe.

A robust design should:

- Establish an end-to-end request deadline.
- Give each downstream call an appropriate timeout within that deadline.
- Cancel work that is no longer needed.
- Avoid holding database connections while waiting on unrelated remote services.
- Propagate request cancellation where supported.
- Return a controlled error when a dependency is unavailable.

Structured concurrency can help express parent-child task relationships and coordinate failures and cancellation. Its API has evolved across Java releases, so use the API appropriate for your target JDK.

4.5 Virtual threads and Spring Boot

For a Spring Boot application, virtual threads can be enabled in supported Spring Boot versions with a configuration property:

```
properties
spring.threads.virtual.enabled=true
```

Verify that your exact Spring Boot release supports this property and that the server and libraries in use are compatible. Enabling virtual threads does not automatically convert every internal executor, scheduler, or third-party library to virtual-thread execution.

Also inspect:

- JDBC driver and connection-pool behavior.
- HTTP client timeouts and connection limits.
- Scheduled jobs and background executors.
- Framework-specific thread-local usage.
- Native libraries and blocking filesystem operations.
- Request admission and overload handling.

The most effective migration usually starts with a measured, I/O-bound endpoint rather than a blanket rewrite of every execution path.

5. HLD capacity planning: An interactive example

Change the workload assumptions below to see how much concurrency Little's Law predicts.

Service concurrency estimator

An illustrative steady-state model, not a load-test result.

Request throughput **1,000 req/s**

100 2,500 5,000

Average request latency **500 ms**

50 ms 1 s 2 s

CPU time per request **10 ms**

1 ms 100 ms 200 ms

Database connection pool size **50**

10 100 200

Average in-flight requests

500

Throughput × latency

CPU cores fully utilized equivalent

10

Before overhead and peak-load margin

Idealized DB service capacity

100 ops/s

Assumes one DB operation per request, taking the full modeled latency

CPU demand at target throughput

10 core-seconds/s

Does not include GC, framework, or scheduling overhead

The average in-flight request count exceeds the configured connection count. This is not automatically a problem: many requests may be doing other work or waiting. Measure database connection wait time and query demand before changing the pool.

A crucial modeling distinction: the database capacity calculation above is valid only if each connection is occupied for the full modeled latency and each request needs exactly one database operation. In

real systems, use actual connection hold time and database-operation frequency instead.

---

6. Performance pitfalls and production considerations

Virtual threads solve one particular resource problem. They do not eliminate other resource constraints.

Pitfall	Why it matters	Mitigation
Unbounded task submission	Too many in-flight tasks can consume heap and overwhelm dependencies	Admission control, bounded queues, load shedding
Database pool exhaustion	Requests wait for a limited number of connections	Tune pool size against DB capacity; measure connection wait time
CPU saturation	More runnable tasks cannot create more CPU capacity	Use bounded compute executors and profile hotspots
Long lock contention	Threads wait for shared state or serialized operations	Reduce critical sections and redesign shared-state access
Native or foreign calls	Some blocking native operations can pin carriers	Profile and inspect native integration
Excessive thread-local state	Per-thread objects can increase memory usage	Minimize large per-thread allocations and clean up state
Missing deadlines	A virtual thread can wait for a long time without a useful result	Timeouts, cancellation, end-to-end deadlines
Dependency overload	More concurrent requests can overwhelm an external service	Semaphores, rate limits, circuit breakers and bulkheads

Observability and troubleshooting

Use the following tools when moving a service to virtual threads:

- **Java Flight Recorder (JFR):** inspect virtual-thread activity, scheduling, and relevant pinning events.
- `jcmd`**:** capture thread dumps, including virtual-thread information using the supported thread-dump options for your JDK.
- **Micrometer and Prometheus:** monitor request latency, throughput, active requests, rejected requests, and resource utilization.
- **HikariCP metrics:** track active, idle, and pending connection borrowers.
- **Distributed tracing:** identify time spent in database queries and downstream HTTP calls.
- **JMH and load testing:** benchmark representative workloads rather than relying on synthetic thread-creation benchmarks.

For Java 21, you can investigate monitor-related pinning that involves blocking operations. In Java 24 and later, JEP 491 substantially changes this behavior by allowing virtual threads to unmount while using `synchronized` methods and blocks. Do not interpret an old JDK's pinning diagnosis as a universal rule for current JDKs.

Oracle Docs +2

---

7. When should an architect choose virtual threads?

Strong fit

- REST APIs that perform blocking I/O.
- Microservices that call multiple downstream services.
- JDBC-based services where requests frequently wait for queries.
- Legacy blocking code that would be expensive to rewrite into asynchronous pipelines.

Evaluate through benchmarking

- Applications with large amounts of synchronized or native code.
- Services with substantial thread-local state.
- Systems already using reactive I/O effectively.
- Workloads with tight memory budgets or very high request fan-out.

Not a substitute for

- CPU-parallel algorithms.
- Database scaling.
- Rate limiting and backpressure.
- Distributed locking or consistency protocols.
- Efficient algorithms and data structures.

8. Interview questions for senior and architect roles

Use these questions to test whether you understand the underlying mechanics rather than just the APIs.

1 How do virtual threads differ from platform threads?

2 Do virtual threads make CPU-bound code faster?

3 Why should virtual threads generally not be pooled?

4 How do you prevent 10,000 virtual threads from overwhelming PostgreSQL?

5 What is virtual-thread pinning, and what changed in Java 24?

6 When would you choose a platform-thread executor over virtual threads?

7 How would you migrate a Spring Boot service safely?

9. Recommended learning and hands-on exercise

For your Java backend and architecture preparation, I recommend studying virtual threads in this order:

1. **Thread fundamentals:** platform threads, Java Memory Model, synchronization, executors, futures, and locks.
2. **Virtual-thread mechanics:** carrier threads, mounting/unmounting, scheduling, interruption, and pinning.
3. **Concurrency control:** semaphores, bounded queues, rate limiting, timeouts, and cancellation.
4. **Production architecture:** Spring Boot, HikariCP, HTTP clients, backpressure, and observability.
5. **Benchmarking:** compare platform-thread and virtual-thread implementations using the same I/O-bound workload and equivalent resource limits.

Start with the official documentation: JEP 444 — Virtual Threads ↗ and Oracle's Virtual Threads guide ↗.

For a practical proof of concept, implement the same Spring Boot endpoint in two configurations: a platform-thread-based request handler and a virtual-thread-based request handler. Use a controllable downstream service with configurable latency, then compare throughput, p95/p99 latency, CPU utilization, memory usage, and database connection wait time under increasing concurrency.

The most important architectural principle is this: **virtual threads make blocking concurrency more scalable; good system design still determines how many requests your application and its dependencies can safely process.**